

高级数据结构第六章习题答案

学号：202533354 姓名：ZHOU MINGYUAN

例题 1

题干：

判断下面哪些是合法的简单图，并说明原因。

G1:

$$V = \{0, 1, 2\}$$

$$E = \{(0, 1), (1, 2), (0, 2)\}$$

G2:

$$V = \{0, 1, 2\}$$

$$E = \{(0, 0), (0, 1), (1, 2)\}$$

G3:

$$V = \{0, 1, 2\}$$

$$E = \{\langle 0, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 2 \rangle\}$$

G4:

$$V = \{0, 1, 2\}$$

$$E = \{(0, 1), (1, 0), (0, 1)\}$$

答案：

- (1) G1 是合法的简单无向图。
- (2) G2 不是简单图，因为含有自环 $(0, 0)$ 。
- (3) G3 是合法的简单有向图；若题目限定“无向简单图”，则它不是无向图。
- (4) G4 不是简单图，因为 $(0, 1)$ 与 $(1, 0)$ 在无向图中表示同一条边，而且 $(0, 1)$ 重复出现。

解析：

简单图不允许自环，也不允许重复边。无向边 (u, v) 与 (v, u) 等价；有向边 $\langle u, v \rangle$ 与 $\langle v, u \rangle$ 方向不同，因此需要先判断图的类型，再检查自环和重复边。

例题 2

题干：

有 5 个顶点。

$$0, 1, 2, 3, 4$$

- (1) 无向完全图最多有多少条边？
- (2) 有向完全图最多有多少条边？

(3) 若一个无向图有 5 个顶点、10 条边，它是不是完全图？

答案：

(1) 无向完全图边数为 $5 \times 4 / 2 = 10$ 。

(2) 有向完全图边数为 $5 \times 4 = 20$ 。

(3) 是。5 个顶点的无向简单图最多只有 10 条边；若已经有 10 条边，则任意两个不同顶点之间都有边。

解析：

无向完全图每一对顶点只对应一条边，所以除以 2；有向完全图中每一对不同顶点有两个方向的有向边，所以为 $n(n-1)$ 。

例题 3

题干：

给定无向图：

$V = \{0, 1, 2, 3, 4\}$

$E = \{(0, 1), (1, 2), (2, 3), (3, 1), (3, 4)\}$

回答：

(1) 0 和 1 是否邻接？

(2) 0 和 2 是否邻接？是否连通？

(3) 0, 1, 2, 3, 4 是否是一条路径？

(4) 1, 2, 3, 1 是否是环路？

(5) 0, 1, 2, 3, 1 是否是简单路径？

答案：

(1) 0 和 1 邻接。

(2) 0 和 2 不邻接，但二者连通，因为存在路径 0-1-2。

(3) 0, 1, 2, 3, 4 是一条路径。

(4) 1, 2, 3, 1 是环路。

(5) 0, 1, 2, 3, 1 不是简单路径，因为顶点 1 重复出现。

解析：

邻接要求两个顶点之间有直接边；连通只要求存在某条路径。简单路径要求除必要的起点终点情况外顶点不重复，而环路要求起点和终点相同。

例题 4

题干：

给定无向图：

$V = \{0, 1, 2, 3, 4, 5, 6\}$

$E = \{(0, 1), (1, 2), (2, 0), (3, 4), (5, 6)\}$

回答：

- (1) 该图是否连通？
- (2) 写出所有连通分量。
- (3) 从 0 开始做一次 DFS，能否访问全部顶点？
- (4) 若执行 `connected()`，DFS 会从哪些顶点启动？0,3,5

答案：

- (1) 该图不连通。
- (2) 连通分量为 $\{0,1,2\}$ 、 $\{3,4\}$ 、 $\{5,6\}$ 。
- (3) 从 0 开始的一次 DFS 只能访问 $\{0,1,2\}$ ，不能访问全部顶点。
- (4) `connected()` 会分别从 0、3、5 启动 DFS。

解析：

一次 DFS 只能覆盖起点所在的连通分量。`connected()` 会按顶点顺序扫描，遇到尚未访问的顶点就启动一次新的 DFS，因此每个连通分量启动一次。

例题 5

题干：

给定有向图：

$V=\{0,1,2,3\}$

$E=\{\langle 0,1\rangle,\langle 1,0\rangle,\langle 1,2\rangle,\langle 2,3\rangle,\langle 3,2\rangle\}$

回答：

- (1) 0 和 1 是否强连通？
- (2) 2 和 3 是否强连通？
- (3) 1 和 2 是否强连通？
- (4) 写出该图的强连通分量。

答案：

- (1) 0 和 1 强连通。
- (2) 2 和 3 强连通。
- (3) 1 和 2 不强连通。
- (4) 强连通分量为 $\{0,1\}$ 和 $\{2,3\}$ 。

解析：

有向图中强连通要求两个顶点互相可达。1 可以到达 2，但 2 只能在 2 和 3 之间互达，不能回到 1，所以 1 和 2 不属于同一个强连通分量。

例题 6

题干：

给定有向图：

$V=\{0,1,2,3\}$

$E = \{ \langle 0, 1 \rangle, \langle 0, 2 \rangle, \langle 1, 2 \rangle, \langle 2, 0 \rangle, \langle 2, 3 \rangle \}$

回答：

- (1) 顶点 0 的入度和出度是多少？
- (2) 顶点 2 的入度和出度是多少？
- (3) 所有顶点的入度之和是多少？
- (4) 所有顶点的出度之和是多少？

答案：

- (1) 顶点 0 的入度为 1，出度为 2。
- (2) 顶点 2 的入度为 2，出度为 2。
- (3) 所有顶点入度之和为 5。
- (4) 所有顶点出度之和为 5。

解析：

每条有向边贡献 1 个出度给起点、1 个入度给终点，因此总入度 = 总出度 = 边数。本图共有 5 条边。

例题 7

题干：

给定有向图：

$$V=\{0,1,2,3\}$$

$$E=\{\langle 0,1\rangle,\langle 0,2\rangle,\langle 1,3\rangle,\langle 2,3\rangle,\langle 3,1\rangle\}$$

- (1) 写出邻接矩阵。
- (2) 根据邻接矩阵求顶点 3 的入度。
- (3) 根据邻接矩阵求顶点 3 的出度。
- (4) 该矩阵是否对称？为什么？

答案：

- (1) 邻接矩阵为：

0 1 1 0

0 0 0 1

0 0 0 1

0 1 0 0

- (2) 顶点 3 的入度为 2。
- (3) 顶点 3 的出度为 1。
- (4) 矩阵不对称，因为这是有向图，例如有 $0\rightarrow 1$ ，但没有 $1\rightarrow 0$ 。

解析：

有向图邻接矩阵中，第 i 行表示从 i 发出的边，行和是出度；第 j 列表示进入 j 的边，列和是入度。

例题 8

题干：

给定无向图：

$$V=\{0,1,2,3,4\}$$

$$E=\{(0,1),(0,2),(1,3),(2,3),(3,4)\}$$

- (1) 写出该图的邻接表。
- (2) 如果从 0 开始 DFS，邻接点按编号从小到大访问，写出访问顺序。
- (3) 如果从 0 开始 BFS，邻接点按编号从小到大访问，写出访问顺序。

答案：

- (1) 邻接表：

0: 1, 2

1: 0, 3

2: 0, 3

3: 1, 2, 4

4: 3

- (2) 从 0 开始 DFS 的访问顺序为 0, 1, 3, 2, 4。

(3) 从 0 开始 BFS 的访问顺序为 0, 1, 2, 3, 4。

解析：

无向图中每条边要写入两个顶点的邻接表。DFS 会沿着当前分支尽量深入；BFS 先访问与起点距离为 1 的顶点，再访问下一层顶点。

例题 9

题干：

给定邻接表：

0: 1, 2

1: 0, 3, 4

2: 0, 5

3: 1

4: 1

5: 2

从顶点 0 开始执行 DFS，邻接点按给定顺序访问。

(1) 写出 DFS 访问顺序。

(2) 画出 DFS 生成树。

(3) 哪些边是树边？

答案：

(1) DFS 访问顺序为 0, 1, 3, 4, 2, 5。

(2) DFS 生成树可以表示为：0 的孩子为 1、2；1 的孩子为 3、4；2 的孩子为 5。

(3) 树边为 (0,1)、(1,3)、(1,4)、(0,2)、(2,5)。

解析：

按给定邻接表顺序，DFS 从 0 先进入 1，再依次访问 3、4；回到 0 后再进入 2，最后访问 5。第一次发现新顶点所经过的边就是 DFS 树边。

例题 10

题干：

仍使用下面邻接表：

0: 1, 2

1: 0, 3, 4

2: 0, 5

3: 1

4: 1

5: 2

从顶点 0 开始执行 BFS。

- (1) 写出 BFS 访问顺序。
- (2) 写出每一步队列中的内容。
- (3) 画出 BFS 生成树。

答案：

- (1) BFS 访问顺序为 0, 1, 2, 3, 4, 5。
- (2) 队列变化：开始 [0]；访问 0 后 [1,2]；访问 1 后 [2,3,4]；访问 2 后 [3,4,5]；访问 3 后 [4,5]；访问 4 后 [5]；访问 5 后 []。
- (3) BFS 生成树边为 (0,1)、(0,2)、(1,3)、(1,4)、(2,5)。

解析：

BFS 使用队列，先入队的顶点先扩展，因此访问顺序按层展开。某个顶点第一次被加入队列时的边就是 BFS 生成树边。

例题 11

题干：

给定图：

0: 1, 2

1: 0, 3

2: 0, 4

3: 1, 5

4: 2, 5

5: 3, 4

从 0 开始，邻接点按编号从小到大访问。

- (1) 写出 DFS 顺序。
- (2) 写出 BFS 顺序。
- (3) 说明两者访问顺序为什么不同。

答案：

- (1) DFS 顺序为 0, 1, 3, 5, 4, 2。
- (2) BFS 顺序为 0, 1, 2, 3, 4, 5。
- (3) 二者不同是因为 DFS 优先沿一条路径深入到底，而 BFS 优先访问当前顶点的所有同层邻接点。

解析：

在该图中，DFS 从 0 到 1、3、5 后继续深入到 4、2；BFS 则先访问 0 的两个邻接点 1 和 2，再访问下一层的 3 和 4。

例题 12

题干：

给定无向连通图：

$$V = \{0, 1, 2, 3\}$$

$$E = \{(0, 1), (0, 2), (1, 2), (1, 3), (2, 3)\}$$

回答：

- (1) 该图有几个顶点？
- (2) 任意生成树应该有几条边？
- (3) 给出一棵生成树。
- (4) 如果在生成树中再加入一条非树边，会发生什么？

答案：

- (1) 该图有 4 个顶点。
- (2) 任意生成树应有 3 条边。
- (3) 一棵生成树可以取 $\{(0, 1), (1, 2), (2, 3)\}$ 。
- (4) 在生成树中再加入一条非树边会形成一个环。

解析：

连通无向图的生成树包含全部 n 个顶点且无环，因此边数一定是 $n-1$ 。树中任意两点之间路径唯一，加入额外边会产生环。

例题 13

题干：

给定带权无向图边集合：

(0, 1, 4)

(0, 2, 3)

(1, 2, 1)

(1, 3, 2)

(2, 3, 4)

(3, 4, 2)

(4, 5, 6)

使用 Kruskal 算法：

- (1) 按权重从小到大排列边。
- (2) 依次判断每条边是否加入。
- (3) 写出最终最小生成树。
- (4) 写出总权重。

答案：

- (1) 按权重排序为：(1, 2, 1)、(1, 3, 2)、(3, 4, 2)、(0, 2, 3)、(0, 1, 4)、(2, 3, 4)、(4, 5, 6)。
- (2) 依次处理：加入 (1, 2)、加入 (1, 3)、加入 (3, 4)、加入 (0, 2)、跳过 (0, 1)、跳过 (2, 3)、加入 (4, 5)。
- (3) 最小生成树为 $\{(1, 2), (1, 3), (3, 4), (0, 2), (4, 5)\}$ 。
- (4) 总权重为 $1+2+2+3+6 = 14$ 。

解析：

Kruskal 每次从全局最小的候选边开始选择，若加入该边会形成环就跳过。6 个顶点的生成树需要 5 条边。

例题 14

题干：

使用同一个图：

(0, 1, 4)

(0, 2, 3)

(1, 2, 1)

(1, 3, 2)

(2, 3, 4)

(3, 4, 2)

(4, 5, 6)

从顶点 0 开始使用 Prim 算法。

(1) 写出每一步加入的顶点。

(2) 写出每一步选择的边。

(3) 写出最终最小生成树。

(4) 比较 Prim 和 Kruskal 的选边方式有什么不同。

答案：

(1) Prim 从 0 开始加入顶点的顺序为 0, 2, 1, 3, 4, 5。

(2) 选择的边依次为 (0, 2, 3)、(2, 1, 1)、(1, 3, 2)、(3, 4, 2)、(4, 5, 6)。

(3) 最小生成树为 $\{(0, 2), (2, 1), (1, 3), (3, 4), (4, 5)\}$ ，总权重为 14。

(4) Prim 从当前树向外选择最小跨边；Kruskal 是全局按边权从小到大选择，并用成环判断决定是否加入。

解析：

Prim 的核心是维护一个已经在树内的顶点集合，每一步选一条连接树内顶点和树外顶点的最小边。

例题 15

题干：

判断下面说法是否正确，并说明理由。

(1) 最小生成树一定给出了任意两个顶点之间的最短路径。

(2) 最短路径问题只关心两个点之间或一个源点到其他点的路径。

(3) 最小生成树关心的是把所有顶点连起来的总成本最小。

(4) Dijkstra 算法可以用来求最小生成树。

答案：

(1) 错误。最小生成树不保证任意两点之间路径最短。

(2) 正确。最短路径问题关注两个点之间，或一个源点到其他点的最短距离。

(3) 正确。最小生成树关注的是连接所有顶点的总边权最小。

(4) 错误。Dijkstra 用于单源最短路径，不是求最小生成树的算法。

解析：

MST 优化的是“全体连通成本”，最短路径优化的是“路径距离”。两类问题目标函数不同，所以算法也不同。

例题 16

题干：

给定带权有向图邻接矩阵，1000 表示无直接边：

0 1 2 3 4

0 0 6 2 1000 1000

1 1000 0 1 5 1000

2 1000 3 0 8 10

3 1000 1000 1000 0 2

4 1000 1000 1000 1000 0

~~0 1 2 3 4~~

~~0 0 5 2 10 12~~

~~1 1000 0 1 5 1000~~

~~2 1000 3 0 8 10~~

~~3 1000 1000 1000 0 2~~

~~4 1000 1000 1000 1000 0~~

$S = \{0, 2, 1, 3\}$

从顶点 0 开始使用 Dijkstra 算法。

(1) 写出初始 distance。

(2) 每一步写出加入 S 的顶点。

(3) 每一步写出 distance 的变化。

(4) 写出从 0 到所有顶点的最终最短距离。

答案：

(1) 初始 distance 为 $[0, 6, 2, \text{INF}, \text{INF}]$ 。

(2) 加入 S 的顺序为 0, 2, 1, 3, 4。

(3) 选择 2 后更新为 $[0, 5, 2, 10, 12]$ ；选择 1 后无更优更新；选择 3 后到 4 仍为 12；最后选择 4。

(4) 最终最短距离为 $[0, 5, 2, 10, 12]$ 。

解析：

Dijkstra 每轮选择当前 distance 最小且尚未确定的顶点，并用它的出边松弛其他顶点。因为所有边权非负，被加入 S 的顶点距离已经确定。

例题 17

题干：

已知 Dijkstra 算法结束后：

distance = [0, 5, 2, 10, 12]

pi = [-1, 2, 0, 1, 3]

0→2→1→3→4

回答：

(1) 从 0 到 4 的最短路径是什么？

(2) 从 0 到 3 的最短路径是什么？

(3) pi 数组的作用是什么？

答案：

(1) 从 0 到 4 的最短路径为 0→2→1→3→4。

(2) 从 0 到 3 的最短路径为 0→2→1→3。

(3) pi 数组用于记录每个顶点在最短路径树中的前驱顶点。

解析：

恢复路径时从终点开始不断查 pi：4 的前驱是 3，3 的前驱是 1，1 的前驱是 2，2 的前驱是 0，逆序后得到完整路径。

例题 18

题干：

给定有向图：

0 → 1，权重 2

0 → 2，权重 5

2 → 1，权重 -10

回答：

(1) 从 0 到 1 的真实最短距离是多少？

(2) Dijkstra 为什么可能先把顶点 1 定死？

(3) 为什么负权边会破坏 Dijkstra 的前提？

答案：

(1) 从 0 到 1 的真实最短距离为 0→2→1，权重 $5+(-10) = -5$ 。

(2) Dijkstra 可能先把顶点 1 以距离 2 加入 S，因为当时 2 比顶点 2 的距离 5 更小。

(3) 负权边会使“后面绕一下更便宜”成为可能，破坏 Dijkstra 中“已确定顶点不会再变小”的前提。

解析：

Dijkstra 的正确性依赖所有边权非负。若存在负权边，已经确定的最短距离可能在之后被重新降低。

例题 19

题干：

给定边集合：

$(0, 1, 6)$

$(0, 3, 7)$

$(1, 2, 5)$

$(1, 4, -4)$

$(3, 2, -3)$

$(2, 1, -2)$

从源点 0 开始，初始：

$\text{dist}[0]=0$

$\text{dist}[1]=\infty, 6, 2$

$\text{dist}[2]=\infty, 11, 4$

$\text{dist}[3]=\infty, 7$

$\text{dist}[4]=\infty, 2, -2$

完成第一轮和第二轮 Bellman-Ford 松弛，写出每轮结束后的 dist 。

答案：

第一轮松弛后 $\text{dist} = [0, 2, 4, 7, 2]$ 。

第二轮松弛后 $\text{dist} = [0, 2, 4, 7, -2]$ 。

解析：

按题目边顺序松弛：第一轮中 $0 \rightarrow 1$ 得 6， $0 \rightarrow 3$ 得 7， $1 \rightarrow 2$ 得 11， $1 \rightarrow 4$ 得 2， $3 \rightarrow 2$ 改为 4， $2 \rightarrow 1$ 改为 2。

第二轮中 $1 \rightarrow 4$ 可继续改为 -2，其余边不再产生更小距离。

例题 20

题干：

给定有向图：

$0 \rightarrow 1$ ，权重 1

$1 \rightarrow 2$ ，权重 -2

$2 \rightarrow 0$ ，权重 -2

$0 \rightarrow 3$ ，权重 5

回答：

- (1) 0,1,2 是否形成负环?
- (2) 为什么存在负环时最短路径可能不存在?
- (3) Bellman-Ford 如何检测负环?

答案:

- (1) 0,1,2 形成负环, 环权重为 $1 + (-2) + (-2) = -3$ 。
- (2) 存在可达负环时, 可以反复绕环使路径代价不断变小, 因此最短路径没有有限最小值。
- (3) Bellman-Ford 在 $n-1$ 轮松弛后, 再检查所有边; 若仍能松弛, 则存在可达负环。

解析:

n 个顶点的不含环简单路径最多只有 $n-1$ 条边。第 n 轮仍能改进, 说明改进来自某个环; 若该环总权重为负, 就会导致距离趋向负无穷。

例题 21

题干:

判断正误并说明理由。

- (1) BFS 一定比 DFS 更快。
- (2) 无向图的邻接矩阵一定对称。
- (3) 有向图中, 邻接矩阵的行和表示入度。
- (4) DFS 和 BFS 都需要 visited 数组。
- (5) Dijkstra 可以处理负权边, 只要没有负环。
- (6) Bellman-Ford 可以处理负权边, 也可以检测负环。
- (7) Kruskal 和 Prim 都是求最小生成树的算法。
- (8) 最小生成树中可以有环。

答案:

- (1) 错误。BFS 和 DFS 的复杂度通常都是 $O(V+E)$, 谁更快取决于图和任务。
- (2) 正确。无向图中 $A[i][j]$ 与 $A[j][i]$ 同时表示同一条边。
- (3) 错误。有向图邻接矩阵的行和表示出度, 列和表示入度。
- (4) 正确。二者都需要 visited 防止重复访问。
- (5) 错误。Dijkstra 一般不能处理负权边, 即使没有负环也可能出错。
- (6) 正确。Bellman-Ford 可以处理负权边, 并能检测负环。
- (7) 正确。Kruskal 和 Prim 都是最小生成树算法。
- (8) 错误。最小生成树本质上是树, 不能有环。

解析:

本题综合检查图遍历、邻接矩阵、最短路径和最小生成树的概念边界。关键是区分“遍历方式”“路径最短”和“连通总成本最小”。

例题 22

题干:

给定无向带权图：

(0, 1, 2)

(0, 2, 5)

(1, 2, 1)

(1, 3, 4)

(2, 3, 2)

(3, 4, 3)

回答：

- (1) 写出邻接表。
- (2) 从 0 开始写出 DFS 顺序。
- (3) 从 0 开始写出 BFS 顺序。
- (4) 用 Kruskal 求最小生成树。
- (5) 该最小生成树的总权重是多少？

答案：

- (1) 邻接表：
0: (1, 2), (2, 5)
1: (0, 2), (2, 1), (3, 4)
2: (0, 5), (1, 1), (3, 2)
3: (1, 4), (2, 2), (4, 3)
4: (3, 3)
- (2) DFS 顺序为 0, 1, 2, 3, 4。
- (3) BFS 顺序为 0, 1, 2, 3, 4。
- (4) Kruskal 选择边 (1, 2, 1)、(0, 1, 2)、(2, 3, 2)、(3, 4, 3)。
- (5) 最小生成树总权重为 $1+2+2+3 = 8$ 。

解析：

Kruskal 按边权从小到大选边，跳过形成环的边。DFS 和 BFS 在本图按编号访问时顺序相同，但原因不同：DFS 是沿分支深入，BFS 是按层扩展。